

Unity C# 프로그래밍 1차 평가 [교사용 정답 및 해설지]

안내: 본 문서는 교사 및 강사용 표준 정답 및 상세 해설집입니다.

[1번] NullReferenceException (Author가 null)

[유형: 객관식]

지문

아래 C# 코드에서 `Start()`가 실행될 때 `NullReferenceException`이 발생하는 원인으로 가장 알맞은 것을 고르시오.

자료(코드)

```
public class Person { public int Age { get; set; } }
public class Book { public Person Author { get; set; } }

public class PublishBook
{
    private string publisher;
    private Genre genre;

    public void Start()
    {
        Book book = new Book();
        int authorAge = book.Author.Age; // ❌ 예외 발생 지점
        Debug.Log(publisher);
        Debug.Log(genre);
    }
}
```

보기

A. `book`이 null이다. B. `Age`가 null이다. C. `publisher`가 null이라서 예외가 난다. D. `Author`가 null이다.

✅ [공식 정답]

- D. `Author`

💡 [상세 해설]

핵심 근거(3줄 요약)

1. `book`은 `new Book()`으로 생성되어 **null이 아니다**.
2. `Author`는 `Person`(참조 타입)이며 초기화하지 않으면 기본값이 **null**이다.
3. `book.Author.Age`에서 **null(Author)**을 역참조하므로 `NullReferenceException`이 발생한다.

오답 포인트(헛갈리는 지점)

- `publisher(string)`도 기본값은 null일 수 있지만, 보통 `Debug.Log(null)`은 그 자체로 예외를 만들지 않는다.
- `genre(enum)`는 값 타입이라 기본값(0)이 존재한다.
- `Age(int)`도 값 타입이라 null이 될 수 없다. (문제는 `Age`가 아니라 `Author`)

해결/예시(필요할 때만)

해결 1: Author 초기화

```
Book book = new Book();
book.Author = new Person { Age = 20 };
int authorAge = book.Author.Age;
```

해결 2: null 체크

```
Book book = new Book();
int authorAge = book.Author?.Age ?? 0;
```

▶ 추가 팁(선택)

[2번] Inspector 창 기본 기능 (Static / Tag / Prefab)

[유형: 참/거짓(부분 점수)]

지문

아래는 **Inspector(검사기) 창**에 대한 설명이다.

각 문장이 참/거짓인지 판정하시오. (문항 특성상 각 문장별 부분 점수가 적용될 수 있음)

보기(문장)

1. 정적 확인란은 개체가 정적으로 유지된다고 Unity Engine에 알리는 데 사용됩니다.
2. tag 속성은 MonoBehaviour 스크립트를 사용하여 한 번에 여러 개체를 찾는 데 사용할 수 있습니다.
3. 검사기 창은 prefab를 수정할 수 있는 기능을 제공합니다.

✅ [공식 정답]

- 1번: 참
- 2번: 참
- 3번: 참

💡 [상세 해설]

핵심 근거(3줄 요약)

1. Inspector의 **Static 체크**는 오브젝트가 런타임에 움직이지 않는다고 알려 **최적화 기능**에 활용된다.
2. **Tag**는 오브젝트 분류용이며 스크립트에서 같은 태그를 가진 오브젝트를 **여러 개 한 번에 검색**할 수 있다.
3. **Prefab**은 선택/Prefab Mode/인스턴스 Overrides를 통해 Inspector에서 **값을 수정하고 원본에 반영**할 수 있다.

문장별 해설(부분 점수 포인트)

- [문장 1] 참

Inspector의 **Static** 설정은 해당 오브젝트를 “정적 오브젝트”로 취급하게 하여 정적 배치, 라이트맵, 오클루전, 내비메시 등 **각종 최적화/베이킹 과정**에 사용된다.

- [문장 2] 참

Tag는 GameObject를 분류하기 위한 속성이고, 스크립트에서 예를 들어 아래처럼 사용할 수 있다.

```
GameObject.FindGameObjectsWithTag("Player");
```

→ 같은 태그를 가진 오브젝트들을 **배열로 한 번에 찾기** 가능

- [문장 3] 참 Prefab을 선택하거나 Prefab Mode로 열면 Inspector에서 **Prefab의 컴포넌트/값을 편집**할 수 있다. 또한 씬의 Prefab 인스턴스를 수정하면 **Overrides**가 생기고, 이를 **Apply**하여 원본 Prefab에 반영할 수도 있다.

최종 정리

- 1번: 참 / 2번: 참 / 3번: 참 → 따라서 **세 문장 모두 ‘참’** 으로 체크한 선택은 타당하다.

[3번] C# 연산자 의미 매칭 (=, ==, !=, ++, +)

[유형: 매칭(연결)]

지문

아래 연산자를 알맞은 의미에 **연결**하시오.

자료

- [문제에 나온 연산자]
=, ==, !=, ++, +
- [의미 후보]
할당 / 같음(비교) / 같지 않음 / 증분(1 증가) / 문자열 연결

✅ [공식 정답]

- != → 같지 않음
- ++ → 증분(1 증가)
- == → 같음(비교)
- + → 문자열 연결
- = → 할당

💡 [상세 해설]

핵심 근거(3줄 요약)

1. `=`는 값을 변수에 **대입(할당)** 하는 연산자다.
2. `==`, `!=`는 두 값이 **같은지/다른지 비교**해 `bool`을 만든다.
3. `++`는 값을 1 증가시키고, `+`는 **문자열이 섞이면 문자열 연결**로 동작한다.

간단 예시(정답 근거)

- `=` : 값을 **대입(할당)**
예) `x = 10;`
- `==` : 두 값이 **같은지 비교**
예) `if (x == 10) { }`
- `!=` : 두 값이 **다른지 비교**
예) `if (x != 10) { }`
- `++` : 값을 **1 증가**
예) `x++;`
- `+` : 문자열이 포함되면 **문자열 연결**(C# / Unity)
예) `"HP: " + hp`

오답 포인트(헛갈리는 지점)

- `=`(할당)과 `==`(비교)를 혼동하기 쉽다.
- `+`는 숫자끼리면 덧셈이지만, 문자열이 하나라도 섞이면 **연결**이 된다.

최종 정리

- ✅ 제시한 연결 결과는 **모두 올바른 매칭**이다.

[4번] Transform에 없는 멤버 오류 수정하기

[유형: 코드 빈칸 채우기(드롭다운 2단계)]

지문

샘플 코드의 22~23행에서 아래 컴파일 에러가 발생합니다.

```
'Transform' does not contain a definition for 'turretMount' ...
```

드롭다운에서 올바른 옵션을 선택해 **오류가 나지 않도록 배열 타입과 생성 타입을 수정**하세요.

자료(코드)

```
using UnityEngine;

public class WeaponControl : MonoBehaviour
{
    [System.Serializable]
    public class Mount
    {
        public Transform turretMount;
        public Transform turretCache;
    }

    public ①[] tMounts = new ②[2];

    public void Start()
    {
        tMounts[0].turretMount = null;
        tMounts[1].turretMount = null;
    }
}
```

보기(드롭다운 후보)

- Transform
- Mount
- GameObject
- WeaponControl

✅ [공식 정답]

- ①: Mount
- ②: Mount → public Mount[] tMounts = new Mount[2];

💡 [상세 해설]

- turretMount는 Transform의 멤버가 아니라, 중첩 클래스 Mount 안에 정의된 필드입니다.
- 그런데 tMounts를 Transform[]로 만들면 tMounts[0]의 타입이 Transform이 되어 tMounts[0].turretMount 같은 접근이 불가능해서 오류가 납니다.
- 따라서 tMounts의 요소 타입은 Mount여야 하며, 생성도 new Mount[2]로 해야 합니다.

▶ 추가 팁(런타임 오류 방지)

[5번] 코드 조각이 ECS(Entities) 라이브러리를 사용하는가?

[유형: 참/거짓(부분 점수)]

지문

아래 코드 조각이 **ECS libraries**(예: `Unity.Entities`)를 실질적으로 사용하는지 평가하시오.
ECS를 사용하는 코드 조각은 **참**, 사용하지 않는 코드 조각은 **거짓**을 선택하시오.

(각 문항별 부분 점수 가능)

보기(코드 조각)

[코드 1]

```
using UnityEngine;
using System.Collections;

public class Fireball : MonoBehaviour
{
    public Rigidbody fireballPrefab;
    public Transform firePosition;
    public float fireballSpeed;
}
```

[코드 2]

```
using Unity.Entities;
using UnityEngine;

public class ShieldComponent : MonoBehaviour
{
    public float Protection;
    public float Size;
}
```

[코드 3]

```
using Unity.Entities;
using UnityEngine;

public class EnemyMovementSystem : ComponentSystem
{
    public Rigidbody Rigidbody;
    public InputComponent Inputcomponent;
}
```

✅ [공식 정답]

- 코드 1: 거짓
- 코드 2: 거짓

- 코드 3: 참

💡 [상세 해설]

핵심 근거(3줄 요약)

1. 코드 1은 `UnityEngine`의 `MonoBehaviour` 기반으로만 작성되어 **ECS(Entities)와 무관**하다.
2. 코드 2는 `using UnityEngine.Entities;`가 선언되어 있으나, 내부적으로 ECS 기능을 전혀 사용하지 않으므로 **실질적인 ECS 사용이 아니다**.
3. 코드 3은 `ComponentSystem`을 상속하고 있으므로 (비록 구버전 API이더라도) **ECS 라이브러리에 의존하는 코드**다.

문항별 해설(부분 점수 포인트)

- **[코드 1] 거짓** `UnityEngine.Entities`를 참조하지 않고, ECS 타입(`IComponentData`, `SystemBase` 등)도 전혀 없습니다. 전통적인 `GameObject` 기반 로직입니다. → ECS 사용 X
- **[코드 2] 거짓 (주의 포인트)** 최상단에 `using UnityEngine.Entities;`가 선언되어 있지만, 클래스 본문 어디에서도 ECS 라이브러리의 클래스나 인터페이스를 사용하지 않았습니다. C#에서 아무 기능도 호출하지 않는 `using` 선언은 컴파일 시 무시되며, 이를 '라이브러리를 사용했다'고 볼 수 없습니다. → ECS 사용 X
- **[코드 3] 참** `ComponentSystem`은 Unity Entities(ECS)의 시스템 베이스 클래스입니다. 이를 상속받았기 때문에 ECS 라이브러리를 실질적으로 사용한 코드입니다. (참고: `ComponentSystem`은 프리뷰 시절의 구버전 API이며, 최신 ECS에서는 `SystemBase`나 `ISystem`을 사용합니다. 또한, 관리형(Managed) 컴포넌트인 `Rigidbody`를 시스템 내부에 직접 참조하는 것은 ECS의 데이터 지향 설계(DOD)에 어긋나지만, '라이브러리 사용 여부'를 묻는 질문이므로 '참'이 맞습니다.) → ECS 사용 O

오답 포인트(헛갈리는 지점)

- 상단에 `using UnityEngine.Entities;` 네임스페이스가 적혀 있다고 해서 무조건 '참'으로 판단하면 **코드 2**에서 틀리게 됩니다. 단순한 선언이 아니라, 코드 내부에서 해당 라이브러리에 속한 타입(`ComponentSystem` 등)을 실질적으로 활용해야 라이브러리를 사용한 것으로 인정됩니다.

최종 정리

- 코드 1: 거짓 / 코드 2: 거짓 / 코드 3: 참

[6번] UI Text에 점수 표시 코드 완성하기

[유형: 드래그&드롭(코드 조각 배치)]

지문

코드 조각을 사용하여 **점수(score)**를 **UI 텍스트로 올바르게 표시**하세요. 올바른 코드 조각을 선택해 아래 코드의 빈칸(3곳)에 배치하여 코드를 완성하십시오.

자료(미완성 코드)

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;

[ ① ]

public class ScoreManager : MonoBehaviour
{
    public int score = 0;

    [ ② ]

    public void Score(int points)
    {
        Debug.Log("Scored points");
        score += points;

        [ ③ ]
    }
}

```

드래그 토큰(선택지)

- using UnityEngine.Text;
- using UnityEngine.UI;
- public text myText;
- public Text myText;
- myText.settext = ("Score: " + score.ToString());
- myText.text = ("Score: " + score.ToString());

드롭존(배치 위치)

- **Z1(①):** using 구문 추가 위치
- **Z2(②):** UI Text 변수 선언 위치
- **Z3(③):** 점수 갱신 후 UI 텍스트 업데이트 위치

✅ [공식 정답]

- Z1(①): using UnityEngine.UI;
- Z2(②): public Text myText;
- Z3(③): myText.text = ("Score: " + score.ToString());

💡 [상세 해설]

- UI 텍스트 컴포넌트 **Text**는 **UnityEngine.UI** 네임스페이스에 있으므로 using UnityEngine.UI;가 필요합니다.
- 변수 선언은 타입명이 대문자인 **Text**여야 하며(text 아님),
- 실제로 화면에 글자를 바꾸려면 **myText.text** 속성에 문자열을 넣어야 합니다(settext 같은 멤버는 없음).

[7번] 문자열 인수를 받아 UI Text를 갱신하는 메서드 선언

[유형: 코드 빈칸 채우기(드롭다운 3단계: 반환형/메서드명/매개변수)]

지문

아래 클래스에서 `OnTriggerEnter(Collider other)` 안에서 `SendMessageToDisplay("...")`를 호출하고 있습니다. 이 호출에서 전달된 **문자열 인수**를 받아 `Text` 컴포넌트의 텍스트를 설정하는 **올바른 메서드 선언**을 완성하세요.

드롭다운 목록에서 알맞은 옵션을 선택해 코드를 완성하십시오.

자료(코드)

```
using UnityEngine;
using UnityEngine.UI;

public class UnlockGate : MonoBehaviour
{
    public Text textToDisplay;

    private void OnTriggerEnter(Collider other)
    {
        if (other.gameObject.CompareTag("Gate"))
        {
            other.gameObject.SetActive(false);
            SendMessageToDisplay("Congratulations! You have unlocked the gate!");
        }
        else
        {
            gameObject.SetActive(false);
            SendMessageToDisplay("Unfortunately, you have broken your key. You tried to unlock something other than a gate.");
        }
    }

    private [①] [②] [③]
    {
        textToDisplay.text = stringToDisplay;
    }
}
```

보기(드롭다운 후보 예)

- ① 반환형: `bool`, `float`, `GameObject`, `int`, `void`
- ② 메서드명: `sendMessageToDisplay`, `SendMessageToDisplay`, `SendMessageToDisplay`, `setmessagetodisplay`
- ③ 매개변수: `()`, `(string message)`, `(string "message")`, `(string stringToDisplay)`, `(string stringtodisplay)`, `(string messageToDisplay)`

✅ [공식 정답]

- ① `void`
- ② `SendMessageToDisplay`
- ③ `(string stringToDisplay)`

💡 [상세 해설]

- `SendMessageToDisplay(...)`처럼 문자열을 전달하므로 매개변수는 `string` 1개가 필요합니다.
- 메서드는 UI 텍스트를 설정만 하고 값을 돌려주지 않으므로 반환형은 `void`가 맞습니다.
- 본문에서 `textToDisplay.text = stringToDisplay;`를 사용하므로, 매개변수 이름도 `stringToDisplay`로 맞춰야 컴파일 에러가 나지 않습니다.

[8번] "Hello World!"를 콘솔에 출력하는 문장

[유형: 객관식]

지문

다음 중 "Hello World!" 메시지를 콘솔 창에 기록하는 문장은 무엇입니까?

보기

- A. `Debug = "Hello World!";`
- B. `Console.Log("Hello World!");`
- C. `Debug.Log("Hello World!");`
- D. `Debug.Log = "Hello World!";`

✅ [공식 정답]

- C. `Debug.Log("Hello World!");`

💡 [상세 해설]

핵심 근거(3줄 요약)

1. 콘솔에 메시지를 출력하려면 보통 `Log(...)` 같은 **메서드 호출** 형태가 필요하다.
2. `Debug.Log(...)`는 문자열을 인수로 받아 콘솔에 출력하는 올바른 호출이다.
3. 나머지는 존재하지 않는 멤버이거나 (`Console.Log`), 대입문 형태라 (`Debug = ..., Debug.Log = ...`) 을 바르지 않다.

오답 포인트(헛갈리는 지점)

- A: `Debug`는 클래스/타입처럼 쓰이는 이름인데, 문자열을 **대입할 대상(변수)** 이 아니다.
- B: C#에는 기본적으로 `Console.Log` 메서드가 없다. (`Console.WriteLine`이라면 가능)
- D: `Debug.Log`는 호출하는 메서드이지, 문자열을 대입하는 **변수/프로퍼티**가 아니다.

[9번] Unity 편집기 창 종류 매칭 (Hierarchy / Scene / Project / Inspector)

[유형: 매칭(연결) / 드래그&드롭]

지문

각 창의 유형을 정의와 연결하시오. (Unity 편집기 기본 UI)

자료(창 유형)

- 검사기 창
- 계층 구조 창
- 프로젝트 창
- 장면 창

자료(정의: 드롭존)

- Z1: 이 창에는 현재 장면에 있는 모든 GameObject의 목록이 포함되어 있습니다.
- Z2: 이 창은 생성 중인 세계에 대한 대화형 보기입니다.
- Z3: 이 창에서 프로젝트에 속한 자산에 액세스하고 관리할 수 있습니다.
- Z4: 이 창에는 연결된 모든 구성 요소와 해당 속성을 포함하여 현재 선택한 GameObject에 대한 세부 정보가 표시되며, 이 창에서 장면에 있는 GameObjects의 기능을 수정할 수 있습니다.

✅ [공식 정답]

- Z1 → 계층 구조 창
- Z2 → 장면 창
- Z3 → 프로젝트 창
- Z4 → 검사기 창

💡 [상세 해설]

핵심 근거(3줄 요약)

1. 계층 구조 창(Hierarchy) 은 현재 씬의 **GameObject** 목록/트리 구조 를 보여준다.
2. 장면 창(Scene) 은 씬을 편집하는 **대화형(인터랙티브)** 뷰다.
3. 프로젝트 창(Project) 은 프로젝트 에셋 관리, 검사기 창(Inspector) 은 선택 오브젝트의 **컴포넌트/속성** 편집을 담당한다.

오답 포인트(헛갈리는 지점)

- "장면 창"은 오브젝트 목록이 아니라 **씬을 직접 보고 배치/수정하는 화면**이다.
- "검사기 창"은 프로젝트 에셋 목록이 아니라, **선택한 대상(오브젝트/에셋)의 세부 설정**을 편집한다.

[10번] null 비교가 가능한 변수 선택하기

[유형: 드롭다운 2단계(선언 + 조건식)]

지문

아래 코드에서 **null** 값을 반환(가질) 수 있는 **올바른 데이터 타입/변수 선언**을 선택해 코드를 완성한 뒤, 드롭다운 목록에서 콘솔에 **"Object is null"** 메시지를 기록하게 만드는 **올바른 변수 이름**을 선택하세요.

자료(코드)

[드롭다운 ①: 변수 선언]

```
void Start()
{
    if ([드롭다운 ②])
    {
        Debug.Log("Object is null");
    }
    else
    {
        projectile = GameObject.FindWithTag("Projectile");
    }
}
```

보기

드롭다운 ① (변수 선언)

- `public bool hasTurned;`
- `public GameObject projectile;`
- `public int score;`

드롭다운 ② (조건식)

- `(projectile == null)`
- `(hasTurned == null)`
- `(score == null)`

✅ [공식 정답]

- 드롭다운 ①: `public GameObject projectile;`
- 드롭다운 ②: `(projectile == null)`

💡 [상세 해설]

- `GameObject.FindWithTag("Projectile")`는 **GameObject(참조 타입)**를 반환하므로, 이를 담는 변수도 `GameObject projectile`이어야 합니다.
- 참조 타입은 `null`이 될 수 있으므로 `projectile == null` 비교가 가능합니다.
- `bool`, `int` 같은 값 타입은 기본적으로 `null`을 가질 수 없어서(`Nullable`을 쓰지 않는 한) `hasTurned == null`, `score == null`은 올바른 선택이 아닙니다.

[11번] rb 변수의 올바른 타입 선택하기

[유형: 코드 빈칸 채우기(드롭다운)]

지문

다음 코드 예제에서 "Cannot implicitly convert type" 오류를 방지하기 위해 **rb** 변수를 선언할 **적절한 데이터 형식**을 고르세요. 드롭다운 목록에서 정답을 선택하여 코드를 완성하세요.

자료(코드)

```
using UnityEngine;
using System.Collections;

public class ExampleClass : MonoBehaviour
{
    public Vector3 teleportPoint;
    public [데이터 타입] rb;

    void Start()
    {
        rb = GetComponent<Rigidbody>();
    }

    void FixedUpdate()
    {
        rb.MovePosition(transform.position + transform.forward * Time.deltaTime);
    }
}
```

보기(드롭다운 후보)

- Vector3
- Collider
- Rigidbody
- CharacterController

✅ [공식 정답]

- 데이터 타입(시험 드롭다운 선택): **Rigidbody**

💡 [상세 해설]

- **GetComponent<Rigidbody>()**는 **Rigidbody 컴포넌트**를 반환합니다. 따라서 **rb**도 같은 타입이어야 대입이 가능합니다.
- 타입이 **Vector3**, **Collider**, **CharacterController**라면 **GetComponent<Rigidbody>()**의 결과를 **rb**에 넣을 수 없어 **형 변환 오류**가 발생합니다.
- 또한 **MovePosition()**은 **Rigidbody** 계열에서 사용하는 이동 함수이므로, **rb**가 **Rigidbody** 타입이어야 호출할 수 있습니다.

참고: Unity 실제 타입명은 보통 **Rigidbody**(b가 소문자)입니다. 드롭다운에 **RigidBody**로 표시된 경우, 시험 UI 표기를 그대로 선택하면 됩니다.

[12번] OR 조건으로 보너스 처리

[유형: 코드 빈칸 채우기(텍스트 입력)]**지문**

코인이 **extralife**와 같거나 **bonus**와 같으면 **lives**가 1 증가하고 **score**가 1000 증가하도록 코드를 완성하세요. 빈 칸(텍스트 상자)에 들어갈 **필요한 문자**를 입력해 조건식을 완성하면 됩니다.

자료(코드)

```
if (coins [ ① ] extralife [ ② ] coins [ ③ ] bonus)
{
    lives += 1;
    score += 1000;
}
```

✅ [공식 정답]

- ①: ==
- ②: ||
- ③: ==

💡 [상세 해설]

- **coins == extralife** : 코인이 **extralife**와 같은지 비교합니다.
- **||** : **OR(또는)** 연산자로, 왼쪽 또는 오른쪽 조건 중 **하나라도 참이면** 전체 조건이 참이 됩니다.
- 따라서 **coins**가 **extralife**이거나 **bonus**이면 보상(**lives**, **score**)이 적용됩니다.

[13번] Inspector에서 변수 안 보이는 이유**[유형: 객관식]****지문**

스크립트를 생성했고 검사기(Inspector)에서 **playerName** 값을 편집하려고 했지만 항목이 보이지 않습니다. 아래 코드 상황에서 **Inspector에 playerName이 표시되지 않는 이유**를 가장 잘 설명하는 옵션을 고르세요.

자료(코드)

```
using UnityEngine;
using System.Collections;

private class PlayerOne : MonoBehaviour
{
    private string playerName;

    void Start()
    {
        Debug.Log("This world will be saved by " + playerName);
    }
}
```

```
}
}
```

보기

- A. 검사기에서 스크립트 옵션을 토글 방식으로 켜야 합니다.
- B. `playerName`을 `public` 변수로 선언해야 합니다.
- C. 검사기에서 객체를 보려면 먼저 객체에 대한 스크립트를 성공적으로 실행해야 합니다.
- D. 검사기에서 변수를 편집할 수 없습니다.

✅ [공식 정답]

- B

💡 [상세 해설]

- Unity의 Inspector는 기본적으로 **직렬화(Serialize)**되는 필드만 표시합니다.
- 코드의 `playerName`은 `private` 필드이므로 기본 설정에서는 **Inspector에 노출되지 않습니다.**
- 따라서 보기 중 가장 적절한 설명은 **B(공개/public으로 선언해야 한다)** 입니다.
- 실무적으로는 아래처럼 두 가지 방식으로 해결합니다.
 - `public string playerName;` 로 변경하거나
 - `private`을 유지하고 싶다면: `[SerializeField] private string playerName;`

참고: 코드에는 `private class PlayerOne : MonoBehaviour`처럼 클래스 접근 제한자가 `private`로 되어 있는데, 이 표기 자체도 Unity 컴포넌트/스크립트 사용 관점에서 어색하거나 문제가 될 수 있습니다. 다만 **이 문항의 핵심은 “필드가 `private`이라 Inspector에 표시되지 않는다”** 입니다.

[14번] 자식 Transform들을 배열로 반환하기

[유형: 코드 빈칸 채우기(드롭다운)]

지문

아래 코드 조각을 확인한 뒤, 메서드 `GetChildren(Transform tr)`가 **어떤 타입을 반환해야 하는지** 결정하세요. 드롭다운 목록에서 올바른 반환 타입을 선택해 코드를 완성하십시오.

자료(코드)

```
public [반환 타입] GetChildren(Transform tr)
{
    int childCount = tr.childCount;
    Transform[] result = new Transform[childCount];

    for (int i = 0; i < childCount; ++i)
    {
        result[i] = tr.GetChild(i);
    }
}
```

```
    return result;
}
```

보기(드롭다운 후보)

- Transform
- List<Transform>
- Transform[]
- void

✅ [공식 정답]

- 반환 타입: Transform[]

💡 [상세 해설]

- 메서드 안에서 Transform[] result 배열을 생성하고, 마지막에 return result;로 배열을 반환합니다.
- 따라서 반환 타입은 단일 Transform이나 void가 아니라 Transform[] 이어야 합니다.
- List<Transform>을 반환하려면 내부에서도 List<Transform>을 만들고 Add()로 채운 뒤 return해야 하므로, 현재 코드와 맞지 않습니다.

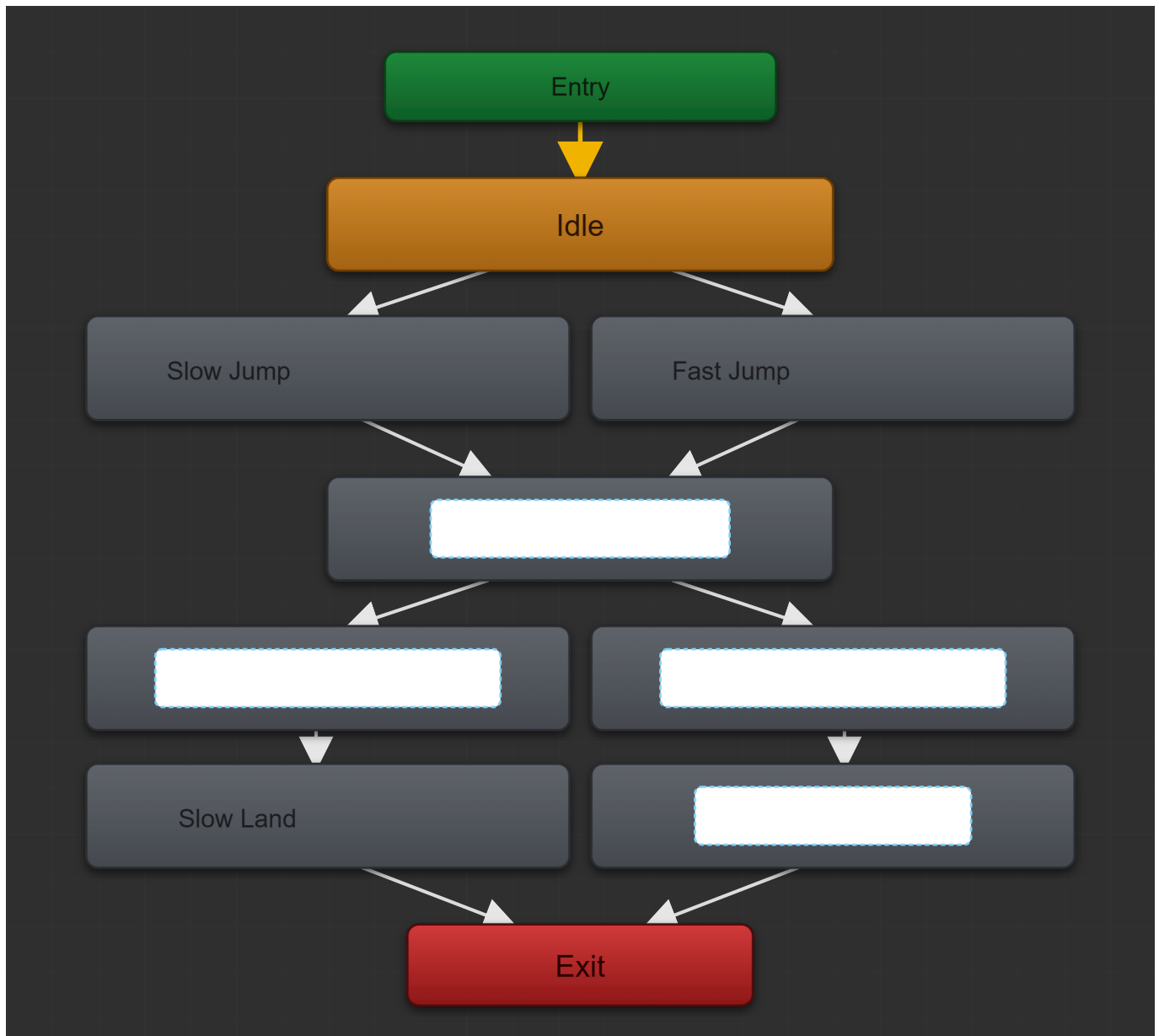
[15번] Animator 점프 상태에 클립 배치하기

[유형: 드래그&드롭(그래픽 매칭)]

지문

캐릭터의 점프 애니메이션 컨트롤러를 구성하려고 합니다.

- 점프는 **Slow Jump** / **Fast Jump** 두 종류가 있습니다.
- 점프 전에는 **Idle** 애니메이션 1개만 사용합니다.
- 점프 정점(최고점) 구간은 **JumpApex** 애니메이션 1개만 사용합니다.
- 제공된 애니메이션 클립을 선택하여, 그래프의 알맞은 상태(State)에 배치해 상태 시스템을 완성하세요.



드래그 토큰(제공 클립)

- JumpApex
- SlowFall
- FastFall
- FastLand

드롭존(배치 위치)

- Z1: JumpApex 상태(가운데, Slow Jump / Fast Jump 아래에 공통으로 연결되는 정점 상태)
- Z2: SlowFall 상태(왼쪽 낙하 상태, JumpApex 이후 왼쪽 가지)
- Z3: FastFall 상태(오른쪽 낙하 상태, JumpApex 이후 오른쪽 가지)
- Z4: FastLand 상태(오른쪽 착지 상태, FastFall 아래)

✅ [공식 정답]

- Z1: JumpApex
- Z2: SlowFall
- Z3: FastFall

- Z4: FastLand

💡 [상세 해설]

- **JumpApex**는 “점프 정점(최고점)” 상태이므로, Slow/Fast 점프가 **공통으로 모이는 가운데 상태**에 배치됩니다.
- 정점 이후에는 낙하가 **속도에 따라 갈라지므로**, 왼쪽은 **SlowFall**, 오른쪽은 **FastFall**이 자연스럽습니다.
- 오른쪽 가지의 착지는 빠른 착지이므로 **FastLand**가 **FastFall** 아래 착지 상태에 들어갑니다.

[16번] OnMouseUp으로 UI 패널 토글 함수 만들기

[유형: 드래그&드롭(코드 조각 배치)]

지문

게임 오브젝트(패널)를 ****켜거나/끄는 함수(이 스크립트에서만 사용 가능)****를 만들어 달라는 요청을 받았습니다. 이 오브젝트는 장면에 있는 3D 오브젝트를 **마우스로 클릭했을 때** 토글 방식으로 제어됩니다.

아래 코드에서 메서드 선언부의 빈칸 3개에 들어갈 **올바른 코드 조각**을 선택해 배치하세요.

자료(코드)

```
using UnityEngine;

public class TurnOnDisplay : MonoBehaviour
{
    public GameObject displayPane;

    private void Start()
    {
        displayPane.SetActive(false);
    }

    [ ① ] [ ② ] [ ③ ]
    {
        if (displayPane.activeInHierarchy == false)
        {
            displayPane.SetActive(true);
        }
        else
        {
            displayPane.SetActive(false);
        }
    }
}
```

드래그 토큰(선택지)

- public
- private

- `void`
- `MouseInfo`
- `OnMouseEnter()`
- `OnMouseUp()`

드롭존(배치 위치)

- Z1(①): 접근 제한자
- Z2(②): 반환형
- Z3(③): 메서드 이름

✅ [공식 정답]

- Z1(①): `private`
- Z2(②): `void`
- Z3(③): `OnMouseUp()`

💡 [상세 해설]

- ****클릭(마우스 버튼을 떼는 순간)****에 호출되는 Unity 메시지 함수가 `OnMouseUp()`입니다.
- “이 스크립트에서만 사용” 조건이므로 접근 제한자는 `public`보다 ****private****가 적절합니다.
- Unity 메시지 함수는 반환값이 없으므로 `void`를 사용합니다.

▶ 추가 팁

[17번] Unity/C# 명명 규칙 판별하기

[유형: 참/거짓(T/F) — 문항별 선택]

지문

아래 코드 조각(1~3) 각각을 보고, **Unity/C# 명명 규칙을 잘 지킨 코드면 ‘참’**, 그렇지 않으면 ****거짓****을 선택하세요.

자료(코드 조각 1)

```
public class playerScript : MonoBehaviour {
    public light playerLight;

    void playerfunction () {
        playerlight.Enabled = !playerlight.Enabled;
    }
}
```

자료(코드 조각 2)

```
Public Class playerScript : MonoBehaviour {
    Public Light PlayerLight;
```

```

Void PlayerFunction () {
    PlayerLight.Enabled = !PlayerLight.Enabled;
}
}

```

자료(코드 조각 3)

```

public class PlayerScript : MonoBehaviour {
    public Light playerLight;

    void PlayerFunction () {
        playerLight.enabled = !playerLight.enabled;
    }
}

```

✅ [공식 정답]

- 코드 조각 1: 거짓
- 코드 조각 2: 거짓
- 코드 조각 3: 참

💡 [상세 해설]

- 코드 조각 1 (거짓)
 - 클래스명 `playerScript`, 함수명 `playerfunction`이 PascalCase가 아님(보통 `PlayerScript`, `PlayerFunction`).
 - `Monobehaviour` 오타(정확히는 `MonoBehaviour`).
 - 타입 `light`는 `Light`가 맞음.
 - 변수 `playerLight`로 선언했는데 `playerlight`로 사용(대소문자 불일치).
 - Unity API 속성은 `Enabled`가 아니라 `enabled`(대소문자 정확히 일치해야 함).
- 코드 조각 2 (거짓)
 - C# 키워드 `public`, `class`, `void`는 소문자가 정답인데 `Public`, `Class`, `Void`로 작성되어 문법적으로도 틀림.
 - 클래스명 `playerScript`가 PascalCase가 아님.
 - `Enabled` 역시 `enabled`가 맞음.
- 코드 조각 3 (참)
 - 클래스명/메서드명 PascalCase, 필드명 camelCase, Unity 타입/베이스클래스 표기(`Light`, `MonoBehaviour`)가 올바르고, `enabled`도 정확합니다.

[18번] Unity에서 사용할 IDE 선택하기

[유형: 객관식]

지문

다음 중 **Unity와 함께 사용할 IDE(스크립트 편집기)** 를 선택할 수 있는 옵션은 무엇입니까? (Unity 설정 화면에서 스크립트 편집기를 지정하는 항목을 고르세요.)

보기

- A. **External Script Editor**
- B. Image application
- C. Revision Control Diff/Merge
- D. Editor Attaching

✅ [공식 정답]

- A

💡 [상세 해설]

- Unity에서 C# 스크립트를 열 때 사용할 IDE는 ****Preferences(또는 Edit > Preferences)****의 **External Script Editor**에서 지정합니다.
- **Image application**은 이미지 파일을 열 프로그램을 고르는 항목이고, **Revision Control Diff/Merge**는 버전 관리 도구의 diff/merge 프로그램 설정입니다.
- **Editor Attaching**은 디버거/에디터 연결 관련 옵션이며 IDE 자체 선택 항목이 아닙니다.

[19번] Random.Range 범위 주석 고르기

[유형: 객관식]

지문

아래 코드 조각에 대한 **올바른 주석**을 고르세요.

자료(코드)

```
// int day = Random.Range(1, 31);
```

보기

- A. **// 1부터 32까지 범위의 숫자를 생성**
- B. **// 날짜 정수를 받을림 수로 변경**
- C. **// 최대 32에 도달할 때까지 날짜를 1씩 증가**
- D. **// 1부터 30까지 범위의 숫자를 생성**

✅ [공식 정답]

- D

💡 [상세 해설]

- Unity의 `Random.Range(int minInclusive, int maxExclusive)`에서 정수 버전은 최댓값이 제외됩니다.
- 따라서 `Random.Range(1, 31)`은 1 이상 31 미만, 즉 1부터 30까지가 실제 결과입니다.

⚠ [실제 자격증 CBT 시험 유의사항]

- **실제 시험 화면 표기:** 실제 Certiport/Pearson VUE 자격증 CBT 화면에서는 시스템 등록 오타로 인해 D번 보기가 // 1부터 31까지 범위의 숫자를 생성으로 오기되어 출제됩니다.
- **출제 오류 원인:** 원래 출제 의도는 1~31일을 생성하는 `Random.Range(1, 32)` 코드였으나 상단 지문의 인자가 (1, 31)로 잘못 입력된 시스템 오류입니다.
- **실전 시험 대처:** 실제 자격증 시험을 보실 때는 상단 코드가 (1, 32)로 적힌 섀 치고 D번을 선택하셔야 공식 채점 시스템에서 정답으로 인정됩니다. (본 학습 플랫폼에서는 기술적 정확성을 위해 D번 보기를 1부터 30까지로 올바르게 교정하여 수록하였습니다.)

[20번] Color 변수 값을 Debug.Log로 출력하기

[유형: 코드 빈칸 채우기(드롭다운)]

지문

아래 `Player` 클래스의 `color` 변수 값을 콘솔에 다음과 같이 출력하려고 합니다.

- 출력 예: `Color: (0.258,0.525,0.956,1)`

드롭다운 목록에서 올바른 옵션을 선택해 `Debug.Log` 코드를 완성하세요.

자료(코드)

```
using UnityEngine;

public class Player : MonoBehaviour
{
    public Color color;

    public void Start()
    {
        // 로그가 여기에 표시됩니다.
    }
}
```

답안 영역(완성해야 할 형태)

```
Debug.Log("Color:" + [드롭다운]);
```

보기(드롭다운 후보)

- `color`
- `Color`
- `new Color(Color.Red)`
- `(0.258,0.525,0.956,1)`

✅ [공식 정답]

- 드롭다운: `color`

💡 [상세 해설]

- 출력해야 하는 대상은 필드 변수 `public Color color;`에 저장된 값이므로 `color`를 선택해야 합니다.
- `Color`는 타입 이름(자료형)이라 값이 아니고,
- `new Color(Color.Red)`는 문제에서 요구한 "변수 `color`의 값"이 아니라 새로운 `Color` 값을 만드는 표현입니다.
- `(0.258,0.525,0.956,1)`은 값의 "표현 예시"일 뿐, 변수 값을 찍는 코드가 아닙니다.

[21번] Unity 명명 규칙 판별하기 2

[유형: 참/거짓(T/F) — 코드 조각별 선택]

지문

아래 코드 조각(1~3) 각각을 보고, **Unity/C# 명명 규칙(대소문자 포함)**을 잘 지킨 코드면 '참', 그렇지 않으면 '**거짓**'을 선택하세요.

자료(코드 조각 1)

```
public class PlaySoundEffect : MonoBehaviour {

    public AudioSource enteringSound;
    public AudioSource leavingSound;

}
```

자료(코드 조각 2)

```
private void OnTriggerEnter(Collider Other)
{
    if (Other.CompareTag("Player"))
    {
        PlaySound(enteringSound);
    }
}
```

자료(코드 조각 3)

```
private void OnTriggerExit(Collider other)
{
    if (other.CompareTag("Player"))
    {
        PlaySound(leavingSound);
    }
}
```

✅ [공식 정답]

- 코드 조각 1: 참
- 코드 조각 2: 거짓
- 코드 조각 3: 참

💡 [상세 해설]

- 코드 조각 1 (참)
 - 클래스명 `PlaySoundEffect`는 PascalCase, 필드명 `enteringSound`, `leavingSound`는 camelCase로 자연스럽습니다.
 - `AudioSource`, `MonoBehaviour` 타입 표기도 올바릅니다.
- 코드 조각 2 (거짓)
 - Unity 메시지 함수는 **대소문자까지 정확히** `OnTriggerEnter`여야 하는데 `ontriggerenter`로 작성되어 호출되지 않습니다.
 - 매개변수 이름은 보통 `other`처럼 camelCase를 쓰는데 `Other`로 되어 있고,
 - `CompareTag` 역시 정확한 멤버 이름은 `CompareTag`인데 `compareTag`로 되어 있어 규칙/코드 모두 어긋납니다.
- 코드 조각 3 (참)
 - `OnTriggerExit`는 Unity 이벤트 메서드 이름이 정확하고,
 - `other.CompareTag("Player")`도 올바른 표기이며,
 - 필드명 `leavingSound`도 camelCase로 적절합니다.

[22번] Unity Transform로 간단 이동 구현하기

[유형: 코드 빈칸 채우기(드롭다운)]

지문

GameObject의 `transform` 구성 요소를 사용해 **간단한 이동 스크립트**를 완성하려고 합니다. 아래 API 정의를 참고하여, 코드의 빈칸(`transform.____( ... );`)에 들어갈 **올바른 메서드**를 드롭다운에서 선택하세요.

자료(API 정의)

- `public void SetPositionAndRotation(Vector3 position, Quaternion rotation);`

- `public Vector3 TransformDirection(Vector3 direction);`
- `public Vector3 TransformVector(Vector3 vector);`
- `public void Translate(Vector3 translation);`

자료(코드)

```
using UnityEngine;

public class ExampleScript : MonoBehaviour
{
    public float speed = 20f;
    private Vector3 move;

    private void Update()
    {
        move = new Vector3(Input.GetAxis("Horizontal"), 0f,
        Input.GetAxis("Vertical"));

        transform.[드롭다운] (move * Time.deltaTime * speed);
    }
}
```

보기(드롭다운 후보)

- `TransformVector`
- `SetPositionAndRotation`
- `Translate`
- `TransformDirection`

✅ [공식 정답]

- 드롭다운: `Translate`

💡 [상세 해설]

- 코드에서 `transform.____(Vector3)` 형태로 **Vector3** 하나를 인수로 받아 위치를 이동시키려면 `Translate(Vector3 translation)`이 정확히 맞습니다.
- `TransformDirection`, `TransformVector`는 **Vector3**를 반환하는 변환 함수라서 `transform.메서드(...);`처럼 “이동” 자체를 수행하지 않습니다.
- `SetPositionAndRotation`은 인수가 (`Vector3`, `Quaternion`) 2개라 현재 호출 형태(인수 1개)와 맞지 않습니다.

[23번] 폴링된 발사체 초기화와 Trigger 충돌 이벤트 선택

[유형: 드래그&드롭(이벤트 함수 배치)]

지문

프로젝트 장르는 탑다운 아케이드입니다. 플레이어가 발사하는 **발사체 프리팹**에는 `IsTrigger = true`로 설정된 **Collider**가 있습니다.

이 발사체는 `Instantiate`로 생성되지 않고 **장면에 Pool**되어 있다가 사용됩니다. 따라서 **풀에서 꺼낼 때 필요한 초기화**가 필요하며, 충돌 처리도 Trigger 방식으로 동작해야 합니다.

아래 코드에서 빈칸 2곳에 들어갈 **올바른 이벤트/함수 이름**을 선택해 배치하세요. (코드는 발사체 프리팹에 붙어 있으며, 플레이어 오브젝트에 붙어 있지 않습니다.)

자료(코드)

```
using UnityEngine;

public class Projectile : MonoBehaviour
{
    private PowerUpManagement PUManage;
    private MeshRenderer meshRenderer;
    private Material instancedMaterial;
    public Color initialColor;

    private bool isAlive = true;
    public int penetration = 1;

    private void [ ㉠ ] ()
    {
        PUManage =
        GameObject.Find("PowerUp_Manager").GetComponent<PowerUpManagement>();
        meshRenderer = GetComponentInChildren<MeshRenderer>();
        instancedMaterial = meshRenderer.material;
        instancedMaterial.SetColor("_TintColor", initialColor);
    }

    private void [ ㉡ ] (Collider other)
    {
        if (isAlive && other.tag != "Player" && other.tag != "CULL")
        {
            isAlive = false;
            PoolManager.Pool["Projectile"].Despawn(transform, penetration);
        }
    }
}
```

드래그 토큰(선택지)

- `Init`
- `Start`
- `Update`
- `FixedUpdate`
- `OnTriggerEnter`
- `OnCollisionEnter`

드롭존(배치 위치)

- Z1(①): 초기화 함수 이름(매개변수 없음)
- Z2(②): 충돌 이벤트 함수 이름(Collider other)

✅ [공식 정답]

- Z1(①): **Init**
- Z2(②): **OnTriggerEnter**

💡 [상세 해설]

- 발사체가 **Pool**에 의해 재사용되면, **Start()**는 “처음 한 번만” 실행되는 경우가 많아(재활성화 때 반복 실행되지 않음) **매번 필요한 초기화에 부적합**할 수 있습니다. 그래서 풀 시스템에서 꺼낼 때 호출하는 커스텀 초기화 함수인 **Init**이 적절합니다.
- 콜라이더가 **IsTrigger = true**이므로 충돌 처리는 **OnCollisionEnter**가 아니라 ****OnTriggerEnter(Collider other)****로 받아야 합니다.

[24번] Unity Rigidbody.AddForce로 바라보는 방향 이동

[유형: 코드 빈칸 채우기(드롭다운 2개)]

지문

이 문제를 해결하려면 **벡터 값과 숫자 값을 곱해 AddForce에 전달**해야 합니다.

- GameObject에 연결된 **Rigidbody**에 **힘을 가해 이동**해야 합니다.
- 개체가 **현재 바라보는 방향**으로 이동해야 합니다.
- 힘의 크기는 **Inspector에서 설정할 수 있는 값**을 사용해야 합니다.

드롭다운 목록에서 올바른 옵션을 선택해 코드를 완성하세요.

자료(코드)

```
using UnityEngine;

public class MovingThings : MonoBehaviour
{
    private float speedOfMotion;
    public float speedForce;
    private static float forceSpeed = 10f;
    private Rigidbody rigidBody;

    void Start()
    {
        speedOfMotion = 10f;
        rigidBody = gameObject.GetComponent<Rigidbody>();
    }

    void FixedUpdate()
```

```
{
    rigidBody.AddForce( [①] * [②] );
}
```

보기(드롭다운 후보)

- ① (방향/벡터)

- `Vector2.facing`
- `120`
- `transform.forward`

- ② (힘의 크기/숫자)

- `speedOfMotion`
- `speedForce`
- `forceSpeed`

✅ [공식 정답]

- ①: `transform.forward`
- ②: `speedForce`

💡 [상세 해설]

- “바라보는 방향”은 `transform.forward`(Vector3)가 가장 정확합니다.
- “Inspector에서 설정 가능한 값”은 `public` 필드인 `speedForce`입니다.
 - `speedOfMotion`은 `private`이고 코드에서 고정값으로 세팅되어 Inspector 조절 대상이 아닙니다.
 - `forceSpeed`도 `private static`이라 Inspector에서 조절하기 어렵고(기본 직렬화 대상도 아님) 요구 조건에 덜 맞습니다.

[25번] 함수 설명의 참/거짓 판별

[유형: 참/거짓(T/F) — 문장별 선택]

지문

다음은 **함수(메서드)**에 대한 설명입니다. 각 문장이 **참인지 거짓인지** 선택하세요. (참고: 각 정답에 대해 부분 크레딧이 적립될 수 있습니다.)

문항

1. `void` 키워드는 함수가 **null 값**을 반환한다는 것을 나타냅니다.
2. 두 개 이상 **다른 유형**의 매개변수를 함수에 전달할 수 있습니다.
3. 먼저 함수를 호출하지 않으면 함수가 실행되지 않습니다.
4. 값을 반환하는 함수는 `return` 키워드를 사용해야 합니다.

✅ [공식 정답]

- 1. 거짓
- 2. 참
- 3. 참
- 4. 참

💡 [상세 해설]

- 1) 거짓: `void`는 "아무 값도 반환하지 않는다"는 뜻입니다. `null`을 반환하는 게 아닙니다.
- 2) 참: 예를 들어 `Func(int count, string name, float speed)`처럼 여러 타입의 매개변수를 동시에 받을 수 있습니다.
- 3) 참: 일반적으로 함수는 **호출(call)** 되어야 실행됩니다(특정 이벤트/엔진 콜백은 엔진이 호출해 주는 형태).
- 4) 참: 반환 타입이 `int`, `float` 등 `void`가 아닌 함수는 실행 경로에서 값을 돌려주기 위해 `return`이 필요합니다.

[26번] 소품(Prop) 데이터 읽기 + 손(Transform)에 장착하기

[유형: 드래그&드롭(코드 블록 순서 배치)]

지문

코드 블록(섞여 있음)

블록 A

```
private void OnEnable() {
```

블록 B

```
public class AttachProp : MonoBehaviour {

    public Transform prop;

    private PropSpecs propSpecs;

    private float damage;
    private float durability;

    private void Awake () {
```

블록 C

```
propSpecs = prop.GetComponent<PropSpecs>();

this.damage = propSpecs.damage;
```

```
this.durability = propSpecs.durability;
}
```

블록 D

```
prop.parent = transform;
prop.position = transform.localPosition;
prop.rotation = transform.localRotation;
}
}
```

✅ [공식 정답]

(올바른 배치 순서)

1. 블록 B
2. 블록 C
3. 블록 A
4. 블록 D

💡 [상세 해설]

- **Awake()**는 장면 로딩 시점에 가까운 초기화 단계라서, **prop**에서 **PropSpecs**를 가져와 **damage/durability**를 먼저 세팅하기에 적합합니다.
- **OnEnable()**은 이 컴포넌트가 활성화될 때 호출되므로, “사용하도록 설정된 경우에만 손에 장착” 조건에 맞게 **부착 로직(Parent/위치/회전)** 을 넣기 좋습니다.
- **prop.parent = transform;**로 손에 붙이고, 이후 위치/회전을 맞춰서 “손에 든 상태”를 만듭니다.

[27번] % 연산으로 짝수 판별 주석 고르기

[유형: 객관식]

지문

아래 코드 한 줄이 수행하는 기능을 가장 정확하게 설명하는 주석을 선택하세요.

자료(코드)

```
if (i % 2 == 0) {
```

보기

- A. // i 정수 변수의 2%가 0과 같은지 확인
- B. // i 변수 정수에서 숫자 2인 숫자의 백분율을 확인
- C. // 정수 변수를 2로 나눌 수 있고 나머지는 0인지 확인

- D. // i 정수 변수 안에 0이 있는지 확인

✅ [공식 정답]

- C

💡 [상세 해설]

- %는 나머지 연산자입니다. `i % 2 == 0`은 "2로 나눈 나머지가 0인지"를 검사하므로 **i가 짝수인지 판별**하는 조건입니다.
- A는 표현이 어색하고("2%"는 퍼센트로 오해 가능), B/D는 의미가 다릅니다.

[28번] static 메서드에서 필드 접근 오류 수정하기

[유형: 드래그&드롭(접근 제한자/키워드 선택)]

지문

샘플 코드의 9행에서 `statFloat` 변수에 오류가 발생합니다. 이는 5행에서 `statFloat` 변수가 **잘못 선언**되었기 때문입니다.

드롭다운(또는 토큰)에서 올바른 키워드를 선택해 5행의 빈칸을 채워 오류를 수정하세요.

자료(코드)

```
public class Example : MonoBehaviour
{
    [ ① ] float statFloat = 0;

    private static void ThisStat()
    {
        statFloat = 1;
    }
}
```

드래그 토큰(선택지)

- public
- private
- protected
- readonly
- const
- static

✅ [공식 정답]

- ①: `static`

💡 [상세 해설]

- `ThisStat()`은 **static 메서드**이므로, 그 안에서 사용되는 멤버도 기본적으로 **static(클래스 멤버)** 이어야 합니다.
- 현재 `statFloat`가 인스턴스 필드라면(static이 없음) static 메서드에서 접근할 수 없어 오류가 납니다.
- 따라서 `statFloat`를 **static**으로 선언해 `static float statFloat = 0;`로 만들면 문제 없이 동작합니다.

▶ 추가 팁

[29번] Scene 뷰에서 개체 배치 관련 설명 참/거짓

[유형: 참/거짓(T/F) — 문장별 선택]

지문

장면 보기(Scene View)에서 개체를 배치하는 작업에 대한 설명입니다. 각 문장이 **참인지 거짓인지** 선택하세요. (부분 크레딧이 있을 수 있습니다.)

문항

1. 개체의 로컬 또는 글로벌 방향을 표시하도록 이동 도구를 구성할 수 있습니다.
2. 변환 도구는 이동, 회전 및 배율 도구를 결합합니다.
3. 꼭지점 맞추기는 선택한 메시의 꼭지점을 장면 그리드에 맞추는 데 사용됩니다.
4. 3D 보기에서 작동하는 경우에만 개체를 이동, 회전 및 배율 조정할 수 있습니다.

✅ [공식 정답]

- 1. 참
- 2. 참
- 3. 거짓
- 4. 거짓

💡 [상세 해설]

- **1) 참:** Unity의 이동/회전/스케일 툴은 핸들 방향을 **Local / Global**로 전환할 수 있습니다.
- **2) 참:** Unity에는 Move/Rotate/Scale을 하나로 묶은 **Transform(또는 Universal) Tool**이 있습니다.
- **3) 거짓:** “꼭지점 맞추기(Vertex snapping)”는 보통 **그리드에 한정되지 않고**, 다른 오브젝트의 **꼭지점(또는 표면)**에 맞춰 정밀 배치할 때 사용됩니다.
- **4) 거짓:** 오브젝트의 Transform은 **Inspector에서도 수치로** 이동/회전/스케일 조정이 가능하므로 “3D 보기에서만 가능”은 틀립니다.

[30번] UI 버튼 이벤트 등록 위치에 따른 동작 판단

[유형: 참/거짓(T/F) — 문장별 선택]

지문

아래 샘플 코드를 검토하세요. 그리고 아래 설명(1~3)이 **참인지 거짓인지** 선택하세요. (참고: 각 정답에 대해 부분 크레딧이 적립될 수 있습니다.)

자료(코드)

```
public class UILightOn : MonoBehaviour
{
    public Image lightAsset;
    public Button button1;
    public Button button2;
    public Button button3;

    void Start()
    {
        button1.onClick.AddListener(LightBulbOn);
    }

    void Update()
    {
        button2.onClick.AddListener(LightBulbOn);
    }

    void OnTriggerEnter2D(Collider2D collision)
    {
        button3.onClick.AddListener(LightBulbOn);
    }

    void LightBulbOn()
    {
        lightAsset.color = Color.green;
    }
}
```

문항

1. `onClick.AddListener`는 `Start` 함수 내부에서 호출되므로 `button1`은 `lightAsset`의 `color`를 녹색으로 바꿉니다.
2. `OnTriggerEnter2D` 함수는 버튼 누름을 감지하는 데 사용해야 하므로 `button3`만 `lightAsset`의 `color`를 녹색으로 바꿉니다.
3. `onClick.AddListener`는 `Update` 함수 내부에서 호출할 수 있으므로 `button2`는 `lightAsset`의 `color`를 녹색으로 바꿉니다.

✅ [공식 정답]

- 1. 참
- 2. 거짓
- 3. 참

💡 [상세 해설]

- 1) 참: `Start()`에서 `button1`에 클릭 리스너가 등록되므로, **button1**을 클릭하면 `LightBulbOn()`이 실행되어 색이 녹색이 됩니다.

- **2) 거짓:** `OnTriggerEnter2D`는 **2D 트리거 충돌 이벤트**이며 버튼 클릭 감지용이 아닙니다. 게다가 이미 `button1`, `button2`도 리스너가 등록되므로 "button3만 바뀐다"는 설명은 틀립니다.
- **3) 참:** `Update()`에서 `button2`에 리스너를 등록하고 있으니, **button2를 클릭하면** 색이 녹색이 됩니다. 다만 `Update()`는 매 프레임 호출되므로 리스너가 **중복으로 계속 추가**되어, 한 번 클릭했는데 `LightBulbOn()`이 여러 번 호출되는 부작용이 생길 수 있습니다(실무에선 보통 `Start/Awake/OnEnable`에서 1회 등록).

[31번] Unity Animator 상태 시스템 전환: 참/거짓

[유형: 참/거짓(T/F) — 문장별 선택]

지문

아래는 상태 시스템(State Machine) 전환에 대한 설명입니다. 각 문장이 **참인지 거짓인지** 선택하세요. (참고: 각 정답에 대해 부분 크레딧이 적립될 수 있습니다.)

문항

1. Entry 노드에서 다른 상태로 전환을 추가하여 어떤 상태에서 상태 시스템이 시작되는지를 제어할 수 있습니다.
2. 기본 상태를 포함하거나 제외하고 상태 시스템을 생성할 수 있습니다.
3. 상태 시스템 내의 각 하위 상태는 별도의 완전한 상태 시스템으로 간주됩니다.
4. 상태 시스템에서 상태 시스템으로 전환할 수 있지만, 상태에서 상태 시스템으로 전환할 수 없습니다.

✓ [공식 정답]

- 1. 참
- 2. 거짓
- 3. 참
- 4. 거짓

💡 [상세 해설]

- **1) 참:** `Entry`에서 특정 상태(또는 서브 상태 머신)로 들어가는 전환을 구성하면 "처음 어디로 시작할지" 흐름을 만들 수 있습니다.
- **2) 거짓:** 상태 머신은 (상태가 존재한다면) 시작 지점이 되는 **기본 상태(Default State)** 개념이 필요합니다. 상태가 있는데도 기본 상태가 "없는" 형태로 운영하는 건 불가능한 쪽에 가깝습니다.
- **3) 참:** **Sub-State Machine(하위 상태 머신)**은 내부에 `Entry/Exit` 등을 가지는 "작은 상태 머신"으로 볼 수 있어, 별도의 상태 시스템처럼 동작합니다.
- **4) 거짓:** **상태(State) → 상태 머신(Sub-State Machine)** 전환도 가능합니다(반대로 상태 머신 → 상태 전환도 가능). "상태에서 상태 시스템으로 전환할 수 없다"는 설명이 틀렸습니다.

[32번] Unity Animator 파라미터 타입에 맞는 Set 함수 연결

[유형: 드래그&드롭(함수 매칭)]

지문

각 Animator 함수(토큰)를 **올바른 매개변수 형태**와 연결하세요. (참고: 각 정답에 대해 부분 크레딧이 적립될 수 있습니다.)

드래그 토큰(선택지)

- SetFloat
- SetInt
- SetTrigger
- SetBool

답안 영역(빈칸)

1.

```
[ ① ] ("Animation", 1);
```

2.

```
[ ② ] ("Animation", .5f);
```

3.

```
[ ③ ] ("Animation", false);
```

4.

```
[ ④ ] ("Animation");
```

✅ [공식 정답]

- ① SetInt
- ② SetFloat
- ③ SetBool
- ④ SetTrigger

💡 [상세 해설]

- 1은 정수 리터럴 → SetInt(string, int)
- .5f는 float 리터럴 → SetFloat(string, float)
- false는 bool → SetBool(string, bool)
- Trigger는 값 없이 "발동"만 시키는 파라미터 → SetTrigger(string)

[33번] Unity Animator.SetBool로 "Attacking"을 false로 설정

[유형: 코드 빈칸 채우기(드롭다운 2개)]

지문

Animator 참조 변수에 대해, 드롭다운을 사용하여 Boolean 매개 변수 **"Attacking"**을 **false**로 설정하세요. 드롭다운 목록에서 올바른 옵션을 선택하여 코드를 완성하십시오.

자료(코드)

```
Animator animator;

void OnTriggerEnter2D(Collider2D collider)
{
    GameObject obj = collider.gameObject;

    if (obj.GetComponent<Player>())
    {
        [드롭다운 ①].SetBool [드롭다운 ②]
    }
}
```

보기(드롭다운 후보)

드롭다운 ① (호출 주체)

- Animator
- animator

드롭다운 ② (인수 형태)

- (Attacking, false);
- ("Attacking", "false");
- ("Attacking", false);
- (Attacking, "false");

✅ [공식 정답]

- ① animator
- ② ("Attacking", false);

💡 [상세 해설]

- **Animator**는 **타입 이름**이고, 실제 메서드를 호출하려면 변수(인스턴스)인 **animator**를 사용해야 합니다.
- **SetBool**의 시그니처는 **SetBool(string name, bool value)** 형태이므로
 - **"Attacking"**은 **문자열**(따옴표 필요),
 - **false**는 **bool 리터럴**(따옴표 없이)이어야 합니다.

[34번] Unity ECS 사용 여부 참/거짓 판별

[유형: 참/거짓(T/F) — 코드 조각별 선택]

지문

참/거짓을 선택하여 각 코드 조각이 **ECS를 사용하는지 여부**를 판단하세요.

코드 조각 1

```
using Unity.Entities;
using UnityEngine;

public class KeyboardComponent : MonoBehaviour
{
    public float Horizontal;
    public float Vertical;
}
```

코드 조각 2

```
using UnityEngine;
using System.Collections;

public class KeyboardScript : MonoBehaviour
{
    public class Wizard
    {
        public int fireballs;
        public int shields;
        public int missiles;
    }
}
```

코드 조각 3

```
using UnityEngine;
using System.Collections;

public class Movement : MonoBehaviour
{
    public float speed;
    public float turnSpeed;

    void Update()
    {
        Movement();
    }
}
```

✓ [공식 정답]

- 코드 조각 1: 거짓
- 코드 조각 2: 거짓
- 코드 조각 3: 거짓

💡 [상세 해설]

- 이 문항에서 “ECS를 사용한다”는 것은 보통 다음처럼 **ECS 구조가 실제로 등장**하는 경우를 말합니다.
 - `IComponentData` / `IBufferElementData` 같은 데이터 컴포넌트
 - `SystemBase` / `ISystem` 같은 시스템
 - `Entity`, `EntityManager`, `Entities.ForEach` 등 엔티티 처리 코드
- 코드 조각 1은 `using Unity.Entities;`가 있지만, 클래스가 `MonoBehaviour` 기반(문항에는 `MonoBehavior` 오타)이고, `IComponentData`/시스템/`EntityManager` 같은 **ECS 구성요소가 전혀 없으므로 ECS 코드라고 보기 어렵습니다.** → 따라서 거짓
- 코드 조각 2는 `MonoBehaviour` 내부에 일반 클래스(`Wizard`)를 선언한 전형적인 OOP 스타일로 ECS와 무관합니다. → 거짓
- 코드 조각 3도 `MonoBehaviour` 기반의 일반 이동 스크립트 형태이며 ECS 요소가 없습니다. → 거짓

[35번] Unity Animator 파라미터 선택: reset으로 돌아가기

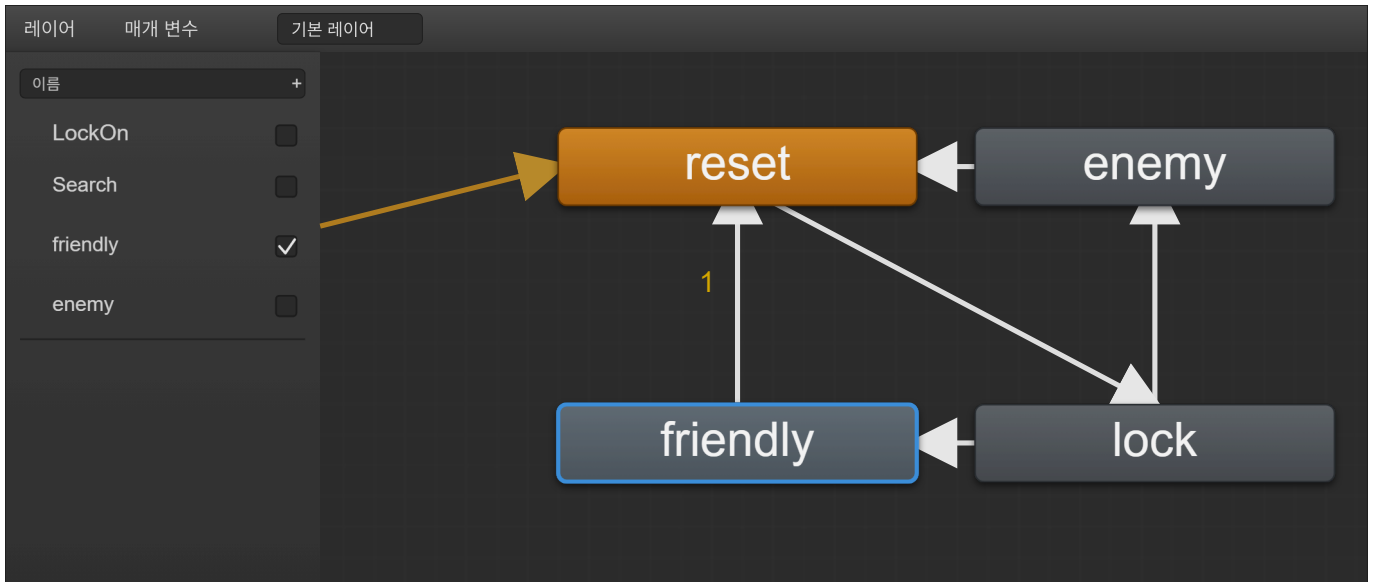
[유형: 문장 완성(드롭다운 2개)]

지문

다음 조건을 만족하도록 문장을 완성해야 합니다.

- 장면은 **재생(Play)** 모드입니다.
- 현재 Animator의 상태는 **friendly** 입니다.
- **Search** 매개변수(**Trigger**) 는 **reset** 상태로 전환하는 데 사용됩니다.

그래픽(Animator 그래프/파라미터 목록)을 참고하여, 드롭다운에서 올바른 옵션을 선택해 문장을 완성하세요.



답안 문장(빈칸)

트리거 [①] 및 Bool을 [②] **false**로 설정하십시오.

보기(드롭다운 후보)

- LockOn
- Search
- Friendly
- Enemy

✅ [공식 정답]

- ① Search
- ② Friendly

💡 [상세 해설]

- 조건에서 **Search Trigger**가 **reset** 상태로 전환하는 데 사용된다고 했으므로 ①은 **Search**입니다.
- 현재 상태가 **friendly**이고, 파라미터 목록에서 ****Friendly Bool**이 true(체크 상태)**로 보이므로, **reset**으로 돌아가기 위해 Bool **Friendly**를 **false**로 내리는 선택이 ②에 해당합니다.
- 참고: Animator 파라미터는 보통 대소문자를 구분합니다. 이 문항은 **드롭다운 표기(Friendly)** 기준으로 선택합니다.

[36번] 발사체 생성 + 전방 속도 부여를 설명하는 주석 2개 고르기

[유형: 다중 선택(2개)]

지문

아래 코드에 대한 주석을 추가하려고 합니다. 코드가 수행하는 기능을 **정확하게 설명하는 주석 2개**를 선택하세요. (2개 선택)

자료(코드)

```
if (Input.GetButtonDown("Fire1")) {
    Rigidbody clone;
    clone = Instantiate(projectile, transform.position, transform.rotation);
    clone.velocity = transform.TransformDirection(Vector3.forward * 10);
}
```

보기

- A. // 원래 개체를 폐기하고 복제본으로 대체합니다.
- B. // <위치>에서 복제본을 인스턴스합니다. 발사체가 적에게 발사되도록 합니다.
- C. // 이 변환의 위치와 회전에서 발사체를 인스턴스화
- D. // 복제된 개체에 현재 개체의 Z 축을 따라 초기 속도를 제공합니다.

✅ [공식 정답]

- C, D

💡 [상세 해설]

- `Instantiate(projectile, transform.position, transform.rotation);` → 현재 오브젝트의 위치/회전에서 발사체를 생성하므로 C가 정확합니다.
- `clone.velocity = transform.TransformDirection(Vector3.forward * 10);`
 - `Vector3.forward`는 로컬 Z+(전방) 방향
 - `TransformDirection`으로 로컬 전방을 월드 방향으로 변환 → 현재 오브젝트의 전방(Z축)으로 속도를 주는 것이라 D가 정확합니다.
- A는 코드에 원본 삭제가 없어서 틀리고, B는 "적에게 발사" 같은 내용이 코드에 보장되지 않아 과장/추측입니다.

[37번] Unity C#에서 컴파일을 막는 비교식 3개 찾기

[유형: 드래그&드롭(오류 비교식 3개 선택)]

지문

아래 코드에는 변수 비교(Comparison) 때문에 컴파일이 실패하는 부분이 있습니다. 보기 중에서 컴파일 오류를 발생시키는 비교식 3개를 골라 답안 영역에 배치하세요.

자료(코드)

```
public class MyClass : MonoBehaviour
{
    [SerializeField]
    List<GameObject> gameObjects;
```



```

Dictionary<string, GameObject> dictionary;
int myInt;
string myString;
List<GameObject> list;
float myFloat;

private void Start()
{
    if (/* [드롭영역 1] */)
    {
        // 작업을 수행합니다.
    }

    if (/* [드롭영역 2] */)
    {
        // 작업을 수행합니다.
    }
}
}

```

보기(드래그 후보)

- `dictionary == myInt`
- `myFloat > myInt`
- `myString == list`
- `dictionary == list`
- `myFloat <= 100`

✅ [공식 정답]

1. `dictionary == myInt`
2. `myString == list`
3. `dictionary == list`

💡 [상세 해설]

- 컴파일 오류가 나는 비교는 "서로 비교할 수 없는 타입끼리" 연산자를 사용한 경우입니다.
 - `Dictionary<string, GameObject>` 와 `int`는 `==`로 비교 불가 → 오류
 - `string` 과 `List<GameObject>`는 `==`로 비교 불가 → 오류
 - `Dictionary<...>` 와 `List<...>`도 서로 다른 컬렉션 타입이라 `==` 비교 불가 → 오류
- 반대로,
 - `myFloat > myInt` 는 숫자 비교로 가능(정수 `myInt`가 `float`로 승격되어 비교)
 - `myFloat <= 100` 도 숫자 비교로 가능(상수 `100`이 `float`로 처리 가능)

[유형: 드롭다운 선택(3개)]

문제

아래 `Update()` 코드의 3개 `if`문은 각각 (1) 누르고 있는 동안, (2) 한 번 눌린 순간, (3) 떼는 순간에 맞춰 로그를 출력해야 합니다. 각 `if` (`Input.[빈칸]`)에 들어갈 올바른 입력 메서드를 드롭다운에서 선택해 코드를 완성하세요. (부분 점수 있음)

참고: 드롭다운 옵션은 모두 `KeyCode.LeftArrow`로 고정되어 있으며, 이 문제는 키 종류가 아니라 입력 메서드 (`GetKey/Down/Up`) 선택이 핵심입니다.

자료(코드)

```
void Update()
{
    if (Input.[드롭다운 ①])
    {
        Debug.Log("Left Arrow key is being held down");
    }

    if (Input.[드롭다운 ②])
    {
        Debug.Log("Up Arrow key was pressed once");
    }

    if (Input.[드롭다운 ③])
    {
        Debug.Log("Down Arrow key was released");
    }
}
```

보기(각 드롭다운 후보)

- `GetKey(KeyCode.LeftArrow)`
- `KeyDown(KeyCode.LeftArrow)`
- `KeyUp(KeyCode.LeftArrow)`

✅ [공식 정답]

- ① `GetKey(KeyCode.LeftArrow)`
- ② `KeyDown(KeyCode.LeftArrow)`
- ③ `KeyUp(KeyCode.LeftArrow)`

💡 [상세 해설]

- `GetKey` : 키를 누르고 있는 동안 매 프레임 `true` → “being held down”
- `KeyDown` : 키를 누른 그 프레임에만 `true` → “pressed once”
- `KeyUp` : 키를 떼는 그 프레임에만 `true` → “released”

[39번] `gameObjects` 목록으로 `Dictionary` 초기화

지문

다음 Unity 스크립트에서 `gameObjects` 목록을 사용해 `dictionary` 변수를 초기화하세요.

- 키(key) 는 각 `GameObject`의 이름(`gameObject.name`) 이어야 합니다.

아래 코드의 `Start()` 내부 `// your code will go here.` 위치에 들어갈 올바른 코드 조각 4개를 골라, 올바른 순서로 배치하세요.

자료(코드)

```
using System.Collections.Generic;
using UnityEngine;

public class MyClass : MonoBehaviour
{
    [SerializeField]
    List<GameObject> gameObjects;

    private Dictionary<string, GameObject> dictionary;

    private void Start()
    {
        // your code will go here.
    }
}
```

보기(코드 조각)

A.

```
foreach (var gameObject in dictionary) {
```

B.

```
gameObjects.Add(gameObject);
```

C.

```
dictionary.Add(gameObject.name, gameObject);
```

D.

```
dictionary = new Dictionary<string, GameObject>();
```

E.

```
foreach (var gameObject in gameObjects) {
```

F.

```
}
```

✅ [공식 정답]

D → E → C → F

완성 코드(정답 적용)

```
private void Start()
{
    dictionary = new Dictionary<string, GameObject>();
    foreach (var gameObject in gameObjects)
    {
        dictionary.Add(gameObject.name, gameObject);
    }
}
```

💡 [상세 해설]

- **dictionary**는 먼저 **생성**해야 **Add()**로 넣을 수 있습니다. → **D**
- 요구사항이 "**gameObjects** 목록을 사용"이므로 **foreach (var gameObject in gameObjects)**가 맞습니다. → **E**
- 키는 **gameObject.name**, 값은 **gameObject** → **C**
- **foreach** 블록 마무리 → **F**
- A는 **dictionary**를 순회하므로 "목록으로 초기화" 목적에 맞지 않고, B는 오히려 목록에 추가하는 코드라 요구사항과 반대입니다.

[40번] Material.SetColor로 기본 셰이더 색상 설정: "_Color" 와 Color.red

다음 API를 사용해 **Unity** 기본 제공 셰이더에서 공통으로 사용하는 **color** 속성 이름과, **빨간색(Color.red)** 값을 설정하는 코드를 완성하세요.

- API: `Material.SetColor(string name, Color value)`
 - `name` : 셰이더의 색상 속성 이름(예: `"_Color"`)
 - `value` : 설정할 **Color** 값

아래 `material.SetColor( __ , __ );`의 두 빈칸에 들어갈 올바른 옵션을 각각 선택하세요.

자료(코드)

```
material.SetColor( __ , __ );
```

보기

(1) name 자리

- A. `_Color`
- B. `"_Color"`
- C. `"Color"`
- D. `Color`

(2) value 자리

- A. `Red`
- B. `Color.red`
- C. `new Color(red)`
- D. `(0,1,0)`

✅ [공식 정답]

- (1) B. `"_Color"`
- (2) B. `Color.red`

✅ 완성 코드:

```
material.SetColor("_Color", Color.red);
```

💡 [상세 해설]

- `SetColor`의 첫 번째 인자는 **문자열(string)**로 셰이더 프로퍼티 이름을 넘겨야 하므로 `"_Color"`처럼 **따옴표 포함**이 맞습니다.
- 기본 제공 셰이더에서 메인 색상 프로퍼티로 흔히 쓰는 이름이 `_Color`(문자열로는 `"_Color"`)입니다.
- `Color.red`는 Unity의 `Color` 구조체가 제공하는 **정적 빨간색 값**입니다.
- `(0,1,0)`은 (R,G,B) 기준으로 **초록색**에 해당하고, 게다가 `Color` 타입으로 그대로 쓸 수 있는 형태도 아닙니다(보통 `new Color(0,1,0)` 형태).